

Loreweaver: Sistema Especialista Baseado em Grafos de Conhecimento e LLMs para o Universo de Hollow Knight

João Victor Oliveira Santos¹, Kevin Gabriel Morais Mangueira¹, Luiz Henrique Santos da Graça¹, Pedro Henrique Paiva Souza¹, Victor Gabriel Menezes da Silva¹

¹Centro de Informática – Universidade Federal da Paraíba (UFPB), João Pessoa – PB – Brasil

{joao.victor.oliveira.santos, kevin.mangueira, lhsg, pedro.paiva, victor.menezes}@academico.ufpb.br

Abstract. This paper presents Loreweaver, an expert system that answers natural-language questions about the lore of the video game Hollow Knight. The knowledge base is modeled as a property graph (Trilha B) stored in Memgraph and populated by an automated pipeline that scrapes the Portuguese Hollow Knight Wiki, extracts subject–relation–object triples with a Large Language Model (LLM) under a closed schema of 9 entity types and 16 relation types, and normalizes and deduplicates the resulting graph through algorithmic rules combined with LLM-based batch entity resolution. At query time, a text-to-Cypher module translates natural-language questions into Cypher, retrieves the relevant neighborhood of the graph, and a GraphRAG step grounds the LLM's final answer in the retrieved facts. From 365 wiki articles the pipeline extracted 1,631 raw triples, refined to 1,282 triples covering 585 unique entities. We discuss the architecture, the trade-offs of migrating the inference engine from a local quantized model to a cloud LLM, and limitations found through the analysis of the produced graph, including a schema-normalization inconsistency that silently reclassifies about 0.3% of edges.

Resumo. Este artigo apresenta o Loreweaver, um sistema especialista que responde a perguntas em linguagem natural sobre a lore do jogo Hollow Knight. A base de conhecimento é modelada como um grafo de propriedades (Trilha B) armazenado no Memgraph e populada por um pipeline automatizado que raspa a Wiki de Hollow Knight em português, extrai triplas sujeito–relação–objeto com um Modelo de Linguagem Grande (LLM) sob um esquema fechado de 9 tipos de entidade e 16 tipos de relação, e normaliza e deduplica o grafo resultante por meio de regras algorítmicas combinadas com resolução de entidades em lote através de LLM. Na consulta, um módulo text-to-Cypher traduz perguntas em linguagem natural para Cypher, recupera a vizinhança relevante do grafo, e uma etapa de GraphRAG fundamenta a resposta final do LLM nos fatos recuperados. A partir de 365 artigos da wiki, o pipeline extrai 1.631 triplas brutas, refinadas para 1.282 triplas cobrindo 585 entidades únicas. Discutimos a arquitetura, os compromissos da migração do motor de inferência de um modelo local quantizado para um LLM em nuvem e limitações encontradas na análise do grafo produzido, incluindo uma inconsistência de normalização de esquema que reclassifica silenciosamente cerca de 0,3% das arestas.

1. Introdução

A extração de informação de bases textuais ricas em narrativa, como wikis de jogos eletrônicos, enfrenta um desafio recorrente de fragmentação de contexto: fatos sobre um mesmo personagem, item ou local aparecem espalhados em dezenas de artigos distintos, com terminologia inconsistente e referências implícitas. O universo do jogo Hollow Knight é um caso extremo desse problema, com uma lore extensa e não linear distribuída por centenas de páginas da Hollow Knight Wiki.

Este trabalho descreve o **Loreweaver**, um sistema especialista que unifica esse conhecimento em um Knowledge Graph (KG) e permite que usuários façam perguntas em linguagem natural sobre a lore, recebendo respostas fundamentadas diretamente no grafo. A solução combina três elementos: (i) um KG modelado como grafo de propriedades, (ii) um LLM usado tanto na fase offline de construção do grafo quanto na fase online de interface com o usuário, e (iii) uma etapa de GraphRAG que ancora as respostas geradas nos fatos recuperados do grafo, mitigando alucinações. O sistema conta ainda com uma interface web de demonstração (chat) para consultas ao vivo.

2. Modelagem do Conhecimento

2.1. Trilha e fontes de dados

Optou-se pela **Trilha B (Grafo de Propriedades)**, com consultas em Cypher, por ser a mais adequada a um domínio denso em relacionamentos heterogêneos entre personagens, locais e itens, e por permitir integração direta com GraphRAG.

A base de dados foi construída por raspagem da Hollow Knight Wiki em português (hollowknight.fandom.com/pt), via API MediaWiki (biblioteca mwclient). O wikitext bruto de cada página é limpo com `mwparserfromhell`: templates, tabelas mecânicas, navboxes, links de imagem e de idioma são removidos, e a marcação restante é convertida para Markdown. Esse processo produziu um corpus de **365 documentos** Markdown, um por artigo da wiki.

2.2. Esquema do grafo

Para evitar poluição semântica e garantir densidade e consistência do grafo, o pipeline de extração impôs ao LLM um esquema fechado, com todos os labels estando em maiúsculo:

- **Rótulos de nós (9 tipos canônicos):** item, localizacao, npc, inimigo, habilidade, chefe, vendedor, grupo, conceito.
- **Propriedade dos nós:** cada nó possui o atributo nome, em minúsculo e sem acentos.
- **Relações (16 tipos canônicos):** contam, derrota, usa, localizado_em, afeta, requer, executa_habilidade, leva_a, vende, dropa, libera, cria, eh_inimigo_de, eh_relatoado_por, eh_membro_de, protege.
- **Propriedades das arestas:** ‘evidencia’ (trecho do texto original que justifica a tripla, garantindo rastreabilidade) e ‘fonte’ (documento e chunk de origem).

O prompt de extração instrui o modelo a escolher o tipo mais específico quando uma entidade ou relação admite mais de uma classificação (ex.: um vendedor de item é tipado como ‘vendedor’, não ‘npc’; a derrota de um chefe que libera uma passagem é

tipada como ‘libera’, não ‘derrota’), e proíbe a invenção de tipos fora da lista, preferindo descartar uma tripla a forçá-la em um tipo incorreto.

3. Arquitetura do Sistema

O pipeline foi implementado inteiramente em Python 3.12, com dependências geridas via Poetry, e é dividido em quatro etapas sequenciais e independentes, cada uma consumindo a saída da anterior em JSON.

3.1. Etapa 1 – Aquisição e limpeza

O script `01_obter_wiki.py` percorre o namespace principal da wiki, ignora redirecionamentos e páginas vazias, e salva cada artigo limpo como Markdown, servindo de corpus para a etapa seguinte.

3.2. Etapa 2 – Extração de triplas

Cada documento é dividido em chunks de até 3.000 tokens (janela de sobreposição de 400 tokens, tokenização via tiktoken) para respeitar o limite de contexto do LLM e evitar perda de atenção em textos longos. Cada chunk é enviado ao modelo com um prompt de extração, solicitando um JSON estruturado de triplas (origem, tipo_origem, relacao, destino, tipo_destino, evidencia).

3.3. Etapa 3 – Refinamento e resolução de entidades

As triplas brutas passam por uma primeira normalização puramente algorítmica: validação de campos obrigatórios, remoção de acentuação e artigos definidos dos nomes de entidade, descarte de valores muito longos (possíveis frases mal extraídas) e aplicação de um dicionário de sinônimos e de relações inversas (ex.: "venceu" → derrota; "liderado_por" inverte origem/destino e vira lidera).

Em seguida, uma etapa de **resolução de entidades por lotes** agrupa nomes que se referem à mesma entidade (ex.: variações de grafia de um mesmo personagem) usando o LLM com temperatura zero, processando lotes de 30 nomes por vez. o progresso é salvo a cada lote, permitindo retomar o processamento em caso de interrupção sem reprocessar entidades já resolvidas. Ao final, as triplas são deduplicadas pela chave (origem, relação, destino).

3.4. Etapa 4 – Carga no Memgraph e GraphRAG

As triplas refinadas são carregadas no Memgraph via protocolo Bolt (driver neo4j para Python). Cada tripla gera nós e uma aresta usando MERGE, com rótulos e tipos de relação sanitizados dinamicamente e validados contra o esquema fechado (tipos fora da lista recebem fallback para ‘conceito’/‘afeta’).

Na consulta, um módulo text-to-Cypher recebe a pergunta do usuário e, com um prompt contendo o esquema completo do grafo, exemplos de consultas corretas e regras rígidas de sintaxe Cypher, gera a consulta correspondente. A consulta é executada no Memgraph e os resultados brutos (fatos recuperados) são injetados em um segundo prompt, que instrui o LLM a responder **apenas** com base nesses dados, a etapa de GraphRAG que ancora a resposta no grafo.

3.4.1. Interface de demonstração

O fluxo de consulta é exposto ao usuário final por meio de uma interface web (HTML/CSS/JS), batizada de *LoreWeaver*, projetada para comunicar visualmente a natureza híbrida do sistema (RAG combinado a um grafo de conhecimento) através do subtítulo fixo "Powered by RAG + Knowledge Graph".

Na tela inicial (*welcome state*), a interface apresenta um cabeçalho de destaque ("Pergunte qualquer coisa sobre Hallownest."), um parágrafo descritivo explicando que o assistente responde com base em uma base de conhecimento estruturada extraída da wiki do jogo, e seis *chips* de sugestão com perguntas pré-definidas. O clique em um chip envia a pergunta imediatamente, reduzindo a barreira de entrada para o usuário durante as demonstrações.

Após o início da conversa, o layout assume o formato de chat convencional, com as mensagens do usuário alinhadas à direita (bolhas com leve tonalidade azulada) e as respostas do assistente à esquerda, acompanhadas de um avatar em formato de sigilo (SVG customizado) com animação sutil de brilho pulsante, reforçando a identidade visual de "oráculo" do sistema. Enquanto a resposta é processada, um indicador de digitação com três pontos em animação piscante comunica ao usuário que o LLM está gerando a resposta. Uma fileira condensada dos chips de sugestão permanece visível acima do campo de entrada, mantendo a descoberta de perguntas possíveis mesmo após o início da conversa.

3.5. Decisão de arquitetura: LLM em nuvem em vez de LLM local

A versão inicial do projeto utilizava um modelo local via Ollama (Mistral-7B-Instruct/Qwen-7B-Instruct, quantizado em 4 bits) para viabilizar execução totalmente offline em hardware de desenvolvimento típico (GPU com 8GB de VRAM). Ao longo do desenvolvimento, a equipe migrou o motor de inferência para a API do **Google Gemini** (gemini-3.5-flash-lite), motivada pela maior qualidade e consistência na extração estruturada de JSON em lotes grandes de texto, ao custo de depender de conectividade e de um limite de taxa da camada gratuita (14 requisições/minuto). Para lidar com esse limite, o módulo `util_comum.py` implementa uma janela deslizante de controle de taxa e retry com backoff baseado no tempo de espera retornado pela própria API em erros 429. O tamanho de 30 entidades por lote na resolução de entidades (Seção 3.3) é herdado dessa fase anterior com modelo local, mantido por já produzir bons resultados com o modelo em nuvem.

4. Demonstração e Resultados

4.1. Volumetria do grafo

A execução completa do pipeline sobre os 365 documentos do corpus produziu **1.631 triplas brutas**. Após a etapa de refinamento (validação, normalização e deduplicação), restaram **1.282 triplas** distintas, cobrindo **585 entidades únicas**, uma redução de aproximadamente 21% em relação ao total bruto, majoritariamente por deduplicação de fatos repetidos entre chunks sobrepostos e por descarte de triplas com evidência vazia ou entidades inválidas.

A distribuição por tipo de relação é dominada por ‘localizado_em’ (676 arestas, 53% do total), refletindo a densidade de relações espaciais no domínio, seguida por ‘afeta’ (123), ‘contem’ (86), ‘usa’ (83) e ‘vende’ (55). Entre os tipos de nó mais frequentes como origem de triplas estão ‘npc’ (224), ‘item’ (223) e ‘chefe’ (218).

4.2. Perguntas de exemplo e respostas

A tabela a seguir ilustra o funcionamento do GraphRAG com triplas efetivamente presentes no grafo:

Tabela 1. Exemplos de perguntas, consultas geradas e respostas do GraphRAG

Elemento	Conteúdo
Pergunta	"Onde o vendedor Sly pode ser encontrado e o que ele vende?"
Cypher gerado	MATCH (v:VENDEDOR)-[r:VENDE]->(i:ITEM) WHERE toLower(v.nome) CONTAINS "sly" RETURN v.nome AS vendedor, i.nome AS item
Fatos recuperados	sly -[vende]-> chave elegante; sly -[vende]-> carapaça robusta.
Resposta do sistema	Sly vende itens como a Chave Elegante e a Carapaça Robusta, ambos adquiridos em sua loja em Dirtmouth.
Pergunta	"Quem o Cavaleiro derrotou entre os chefes finais do jogo?"
Fatos recuperados (amostra)	cavaleiro -[derrota]-> receptaculo puro; cavaleiro -[derrota]-> radiancia absoluta; cavaleiro -[derrota]-> cavaleiro vazio.
Resposta do sistema	Entre os chefes finais, o Cavaleiro derrota o Cavaleiro Vazio, o Receptáculo Puro e, na rota de final alternativo, a Radiância Absoluta.
Pergunta	"Qual rota leva da Encruzilhada Esquecida a outros locais próximos?"
Fatos recuperados (amostra)	poço -[leva_a]-> encruzilhada esquecida; bonde -[leva_a]-> ninho profundo.
Resposta do sistema	O poço conecta a superfície à Encruzilhada Esquecida; a partir dela, o bonde dá acesso ao Ninho Profundo e à Borda do Reino.

4.3. Limitações

Três limitações foram identificadas:

(1) Perda de atenção em contextos longos. O modelo ocasionalmente ignora relações sutis quando o chunk de texto é extenso, mesmo dentro do limite de 3.000 tokens, especialmente em artigos com muitas subseções.

(2) Balanceamento do tamanho de lote na resolução de entidades. Lotes maiores reduzem o número de chamadas ao LLM, mas aumentam o risco de agrupamentos incorretos (falsos positivos) entre entidades com nomes parecidos; lotes menores são mais precisos, porém mais lentos e sujeitos ao limite de taxa da API.

(3) Inconsistência entre o dicionário de normalização e o esquema final do grafo. A análise do grafo carregado revelou que 4 das 1.282 triplas refinadas (cerca de 0,3%) carregam relações que não pertencem à lista de 16 tipos canônicos usada na carga do Memgraph.

Na prática, essas arestas não são descartadas: o mecanismo de fallback da Etapa 4 as recarrega como ‘afeta’, uma relação genérica, causando perda silenciosa de precisão semântica sem gerar erro visível. Isso ocorre com os termos ‘lidera’ e ‘origina_se_em’, que são alvos de sinônimos na Etapa 3 mas não constam da lista canônica de relações.

5. Conclusão e Trabalhos Futuros

O Loreweaver demonstra a viabilidade de combinar IA simbólica (a precisão e rastreabilidade do grafo de propriedades no Memgraph) com IA generativa (a flexibilidade de um LLM como interface de linguagem), mitigando alucinações ao restringir a geração de respostas aos fatos efetivamente recuperados do grafo. O pipeline completo, da raspagem da wiki à resposta em linguagem natural, processou 365 artigos e consolidou um grafo de 585 entidades e 1.282 relações tipadas, com rastreabilidade textual por meio da propriedade evidência.

A análise dos resultados também evidenciou o valor de auditar o grafo já carregado, e não apenas as etapas intermediárias do pipeline: a inconsistência entre sinônimos de relação e esquema canônico (Seção 4.3) só se tornou visível ao inspecionar a distribuição final de tipos de aresta no banco.

Como trabalhos futuros, propõe-se: (i) corrigir a inconsistência de esquema identificada, unificando o dicionário de sinônimos ao conjunto canônico de relações; (ii) implementar embeddings vetoriais nos nós do grafo para uma recuperação híbrida (vetorial + Cypher), reduzindo a dependência de correspondência exata de nomes; e (iii) expandir a resolução de entidades com técnicas de similaridade de string como pré-filtro algorítmico antes do agrupamento por LLM, reduzindo o número de lotes necessários.

Referências

- Edge, D. et al. (2024) "From Local to Global: A Graph RAG Approach to Query-Focused Summarization", arXiv preprint arXiv:2404.16130.
- Lewis, P. et al. (2020) "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", *Advances in Neural Information Processing Systems (NeurIPS)*, v. 33, p. 9459–9474.
- Memgraph (2024) "Memgraph Documentation", <https://memgraph.com/docs>, novembro.
- Francis, N. et al. (2018) "Cypher: An Evolving Query Language for Property Graphs", *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, ACM, p. 1433–1445.
- Google (2025) "Gemini API Documentation", <https://ai.google.dev/gemini-api/docs>, outubro.
- Gage, B. (2023) "mwparserfromhell: A Python Parser for MediaWiki Wikicode", <https://github.com/earwig/mwparserfromhell>.